How Airbnb used Deep Learning to rank listings (and what you can learn from it)

Lessons from Airbnb's AI experiments, including what worked, what failed, and why simplicity often wins.

JOSE PARREÑO GARCIA
MAR 29, 2025

13 2 8 Share

This week I came across a fascinating paper from the Airbnb engineering team, where they detail how they used deep learning to rank accommodations on their main search page. I initially thought it would be just another fancy architecture paper—but I was blown away by how practical their real-world lessons are. This paper is a great guide for those starting in their journey of applying deep learning, specifically for hotel ranking (it is a bit long but very readable).

One of the things I really enjoyed is that they describe their failures in their journey (not many companies share these). I will not cover these in this post as I really want to focus on what I feel are the key highlights.

If you are interested in the main takeaways from their deep-learning journey, let's dive in!

Where can you find the paper?

You can find the Applying Deep Learning to Airbnb Search in arvix.

This paper was released in 2018 and from the abstract you understand that Airbnb's goal with this paper is to describe *"the work done in applying neural networks in an attempt to break out of that plateau"* (being plateau where they landed after years of applying classical machine learning approaches).

In this paper distillation, we will cover:

1. Start simple with Deep Learning architectures
2. Embeddings are cool, but they can't solve everything
3. Applying deep learning feature engineering best practices
4. Deep Learning explainability (what worked and what didn't)

Start simple with Deep Learning architectures

If you think applying deep learning is just about throwing all your data into TensorFlow and letting it work magic… think again.

When they first started replacing their Gradient Boosted Decision Tree (GBDT) model with deep learning, they didn't see instant gains. In fact, their first attempts performed no better than GBDT. But instead of giving up (or blindly adding complexity), they iterated through four key experiments to find what worked.

What Airbnb tried (and what actually worked)

(b) Booking gains online

Figure 2: Relative gains across models

Experiment	What They Did	Results
1️⃣ Simple NN	⚫ A one-hidden-layer NN trained with the same features as GBDT. ⚫ Optimized for regression loss (point-wise Learning-to-Rank).	📉 No improvement over GBDT, but proved that deep learning could at least match traditional models.
2️⃣ LambdaRank NN	⚫ Instead of regression loss, optimized for pair-wise ranking (LambdaRank) using nDCG loss. ⚫ Trained to directly improve rankings rather than predict bookings.	📈 Significant booking gains! Ranking loss was a better objective than regression.
3️⃣ Stacked Models (GBDT + Factorization Machines + NN)	⚫ Combined GBDT, Factorization Machines (FM), and NN into one massive hybrid model.	⚠️ Slight booking improvement but massive complexity. Harder to train, maintain, and deploy.
4️⃣ Scaling Up: Bigger NN + More Data	⚫ Same as LambdaRank NN, but with: 🔷 10x more training data 🔷 Deeper NN (more layers)	🚀 Best results! Higher bookings, simpler architecture, easier to maintain.

Table: Jose Parreño • Source: AirBnB paper • Created with Datawrapper

The winning scenario (#4) might sound like just *"bigger computers, bigger networks, and more data"*, but, it was smarter than that. Scaling up the NN worked because Airbnb also kept feature engineering simple, focusing on core listing attributes while selectively enriching the model with personalisation signals like Smart Pricing and user viewing history. This reinforces the principle of *"better data beats bigger models"*.

Their learning takeaway

> *"Don't be a hero."*

Start simple. Validate what works. **Scale complexity only when needed.**

Embeddings are cool, but they can't solve everything

The rise of neural networks brought with it a new concept of dimensionality reduction: embedding. One of the main properties of embeddings is how well they work with high cardinality features (words in NLP, sparse users in recommender systems or tokens in LLMs).

So Airbnb thought something like:

"Listings have unique IDs and its a high cardinality space. Similar properties (price, location, amenities) should naturally cluster in an embedding space. Why not use embeddings?"

However, when they tried using property_id through embeddings, the results were not good. The model didn't just struggle; it completely overfit.

What happened and why?

The results showed that the models using embeddings for “property_id” massively overfitted the results (ie, amazing in the training set but appalling in the test set).

You can actually see this in the chart below, where the non-dotted lines where super far apart from each other.

Figure 5: Learning curve with listing id feature

This overfitting happened because embeddings need massive amounts of data to converge. In NLP, video, and audio, data is virtually infinite (you can train on the entire internet, which is what LLMs do…)

But property bookings? ***A single listing can only be booked once per day.***

This means that, even the most popular properties ***max out at 365 bookings per year.*** That is nowhere near enough data to train a robust embedding model, so the network simply memorised patterns instead of generalising.

See? Applying deep learning wasn't so easy after all!

Applying deep learning feature engineering best practices

Deep learning comes with the promise of automating feature engineering (ie, you throw in the data and the forward and back propagation figures out weights, linear, non-linear relationships and so on). However, this is only true if you help the **NN** with good data. And by good data I quote the paper: *"ensuring the features comply with certain properties so that the NN can do the math effectively by itself"*. Let's see 3 examples of what Airbnb did:

1. Feature normalisation

2. Feature smoothing
3. Feature engineering optimised for embeddings

Feature normalisation - Ensure that Neural Nets receive similar range of starting values.

Neural networks struggle when features have wildly different value ranges.

If one feature ranges from 0 to 1 (imagine a ratio against the cheapest price), while another ranges from 1,000 to 10,000 (imagine number of reviews), the massive difference in scale can throw your neural network's training process off balance. Without really getting into details, here are some reasons why these ranges work against the NN:

1. **Overpowering gradients.** Think of gradient-based optimisation (like back-propagation) as a tug-of-war between each feature's errors and weights. Features with larger numerical ranges can pull harder on the rope, overshadowing features in smaller ranges. This makes your network pay more attention to the big numbers.
2. **Slow or unstable training.** Because one feature takes huge jumps in weight updates while the other barely moves, your neural network might take a strange path to convergence (if it even converges at all).

3. **Loss of nuance**
   If a network heavily weights the big-range feature, smaller-range features become an afterthought. This can kill performance if the small-range feature actually holds important predictive information.
4. **Vanishing or exploding activations.** Some activation functions (like sigmoid or tanh) can saturate or blow up when inputs are extremely large, causing training to stagnate or produce wildly swinging gradients. This further disrupts stable learning.

That is why feature normalisation is critical. Industry best practice is to scale everything to a uniform range, like [-1,1] or [0,1], with the median mapped to 0. So, how do you handle features that follow very different distributions?

This is where Airbnb got "creative":

- In case the feature distribution resembles a normal distribution, we transform it by $(feature_val - \mu)/\sigma$, where μ is the feature mean and σ the standard deviation.
- If the feature distribution looks closer to a power law distribution, we transform it by $log(\frac{1+feature_val}{1+median})$.

Feature smoothing - Help the Neural Network converge faster

Airbnb noticed something interesting when analysing their 2-layer neural network: *"the outputs of the layers get progressively smoother in terms of their distributions"* This isn't a coincidence. We know that neural networks tend to converge to smoother distributions over time.

The problem comes when you input features that aren't smooth to begin with. The forces the model to work harder to learn stable and smooth patterns. Below you can see the smooth patterns from *"the values from the hidden layers, [...], and the transform log(1 + relu_output) applied"*.

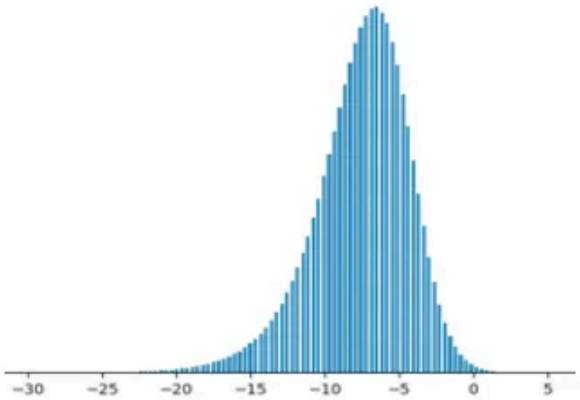


Figure 8: Distribution of output layer.

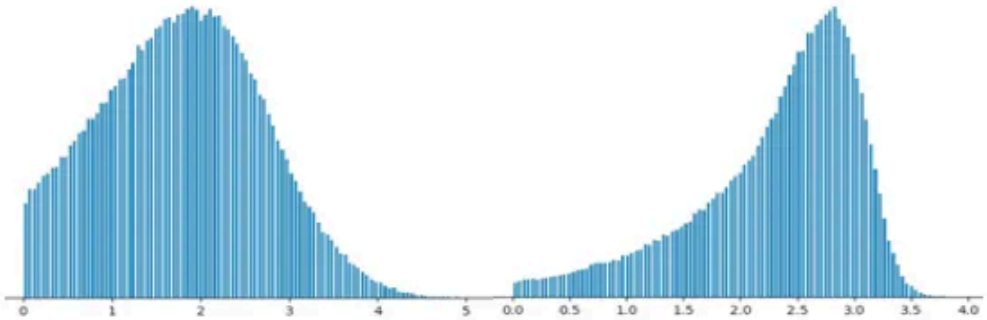


Figure 9: Example distributions from second hidden layer.

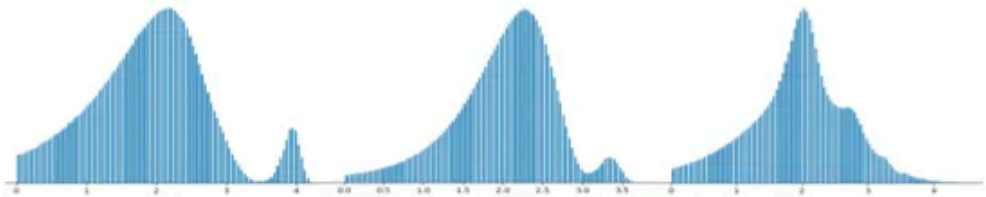


Figure 10: Example distributions from first hidden layer.

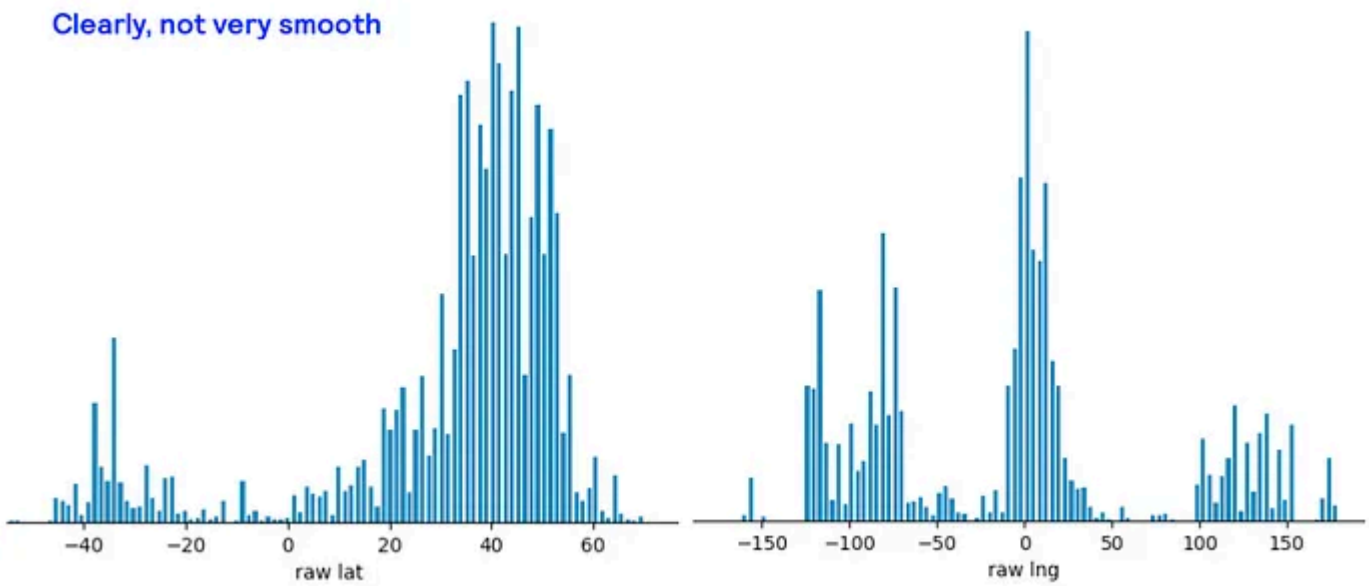
So Airbnb asked:

1. How do we make our features smoother?
2. How can we verify that smoother features actually improve generalisation?

How do we make our features smoother - specifically for *distance*?

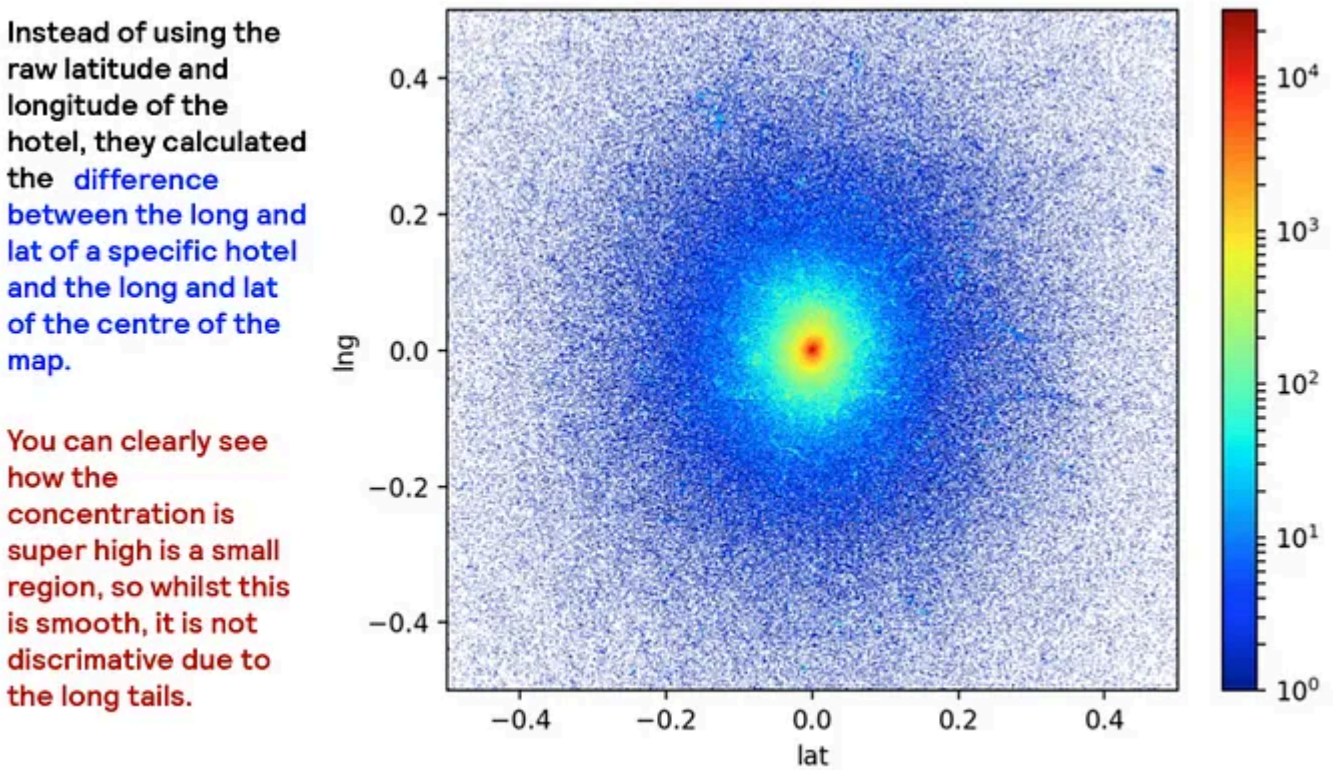
Simple features, like price, can often be smoothed using normalisation (as discussed earlier).

But for more complex data, normalisation alone isn't enough. Take location, for example. Airbnb represents it using latitude and longitude; but, raw latitude and longitude values don't naturally form a smooth feature space.

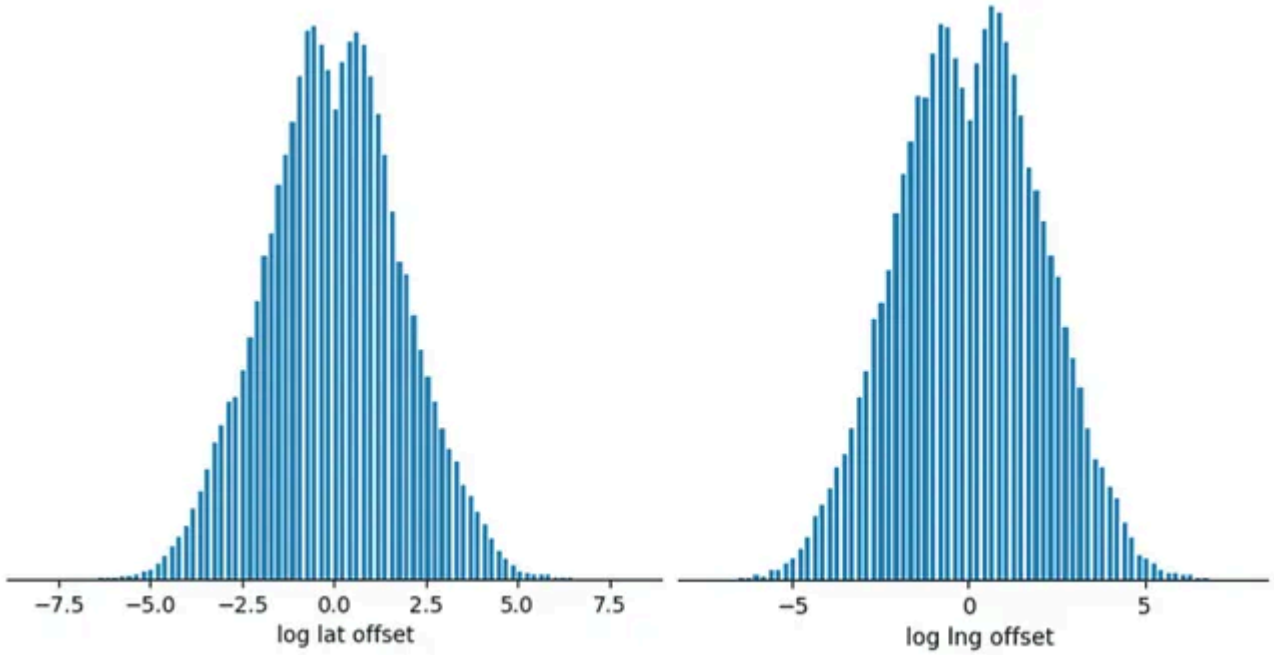


Step 1. Checking that raw lat and long wasn't a smooth feature

So, Airbnb experimented with expressing location relative to the centre of the user's search area. In other words, the *latitude of a hotel - the latitude of the centre of the map*. This made the feature more dense and structured (which was a step in the right direction) but it still wasn't smooth enough. Below you can see the 2D heatmap of the lat and long offsets and how it's smooth but way to concentrated in a small space.



Finally, they decided to take the log of these relative offsets. This transformation compressed large distances while spreading out smaller ones, leading to a smoother distribution.



The result was a transformed location feature that followed a much more natural, continuous distribution, making it easier for the neural network to learn.

How can we verify that smoother features actually improve generalisation?

Option 1. Manually inspect the distribution of the weights of the output (or even the hidden) layers. Basically, plot again the Figs 8, 9 & 10 above.

❌ The issue is that you can only verify this for your training data. In other words, your weights could be smooth for the data you have seen, but you don't know what it would look like on new data.

Option 2. Stress-test the model with unseen values.

One way to validate generalisation is to force the model to handle values it rarely sees in training.

✅ For example, Airbnb artificially scaled up feature values (e.g., 2x or 3x the accommodation price) to see if the model still produced reasonable rankings. If the ranking quality remained stable, it was a sign that feature smoothing helped the model handle new, unseen data better.

💡 **Takeaway:** If your model is struggling to generalise, take a closer look at your features. Sometimes, the right transformation makes all the difference.

Feature engineering optimised for embeddings

Smoothing raw geo-location data is definitely useful, but it **doesn't tell the full story**. Travellers don't just care about coordinates; they care about what is nearby. For

example, I am sure you yourself are interested in things like:

- Is there a metro station?
- Is it a safe neighbourhood?
- Are there restaurants, parks, or museums nearby?

Raw latitude and longitude won't answer these questions. So Airbnb had to get creative.

The solution is so neat, that to do justice to it, I can only copy paste what they wrote in their paper: *"We created a new categorical feature by taking the city specified in the query and the level 12 S2 cell corresponding to a listing, then mapping the two together to an integer using a hashing function. For example, given the query "San Francisco" and a listing near the Embarcadero, we take the S2 cell the listing is situated in (539058204), and hash {"San Francisco", 539058204} => 71829521 to build our categorical feature. These categorical features are then mapped to an embedding, which feed the NN. During training, the model infers the embedding with back propagation which encodes the location preference for the neighborhood represented by the S2 cell, given the city query"* Honestly, beautiful. (Disclaimer, I have not tried in the past to encode the world in level 12 S2 cells, so I have no clue if this is an expensive exercise or not).

The heatmap shows the embedding values learnt for the query San Francisco. The higher the value, the higher the expected booking preference.

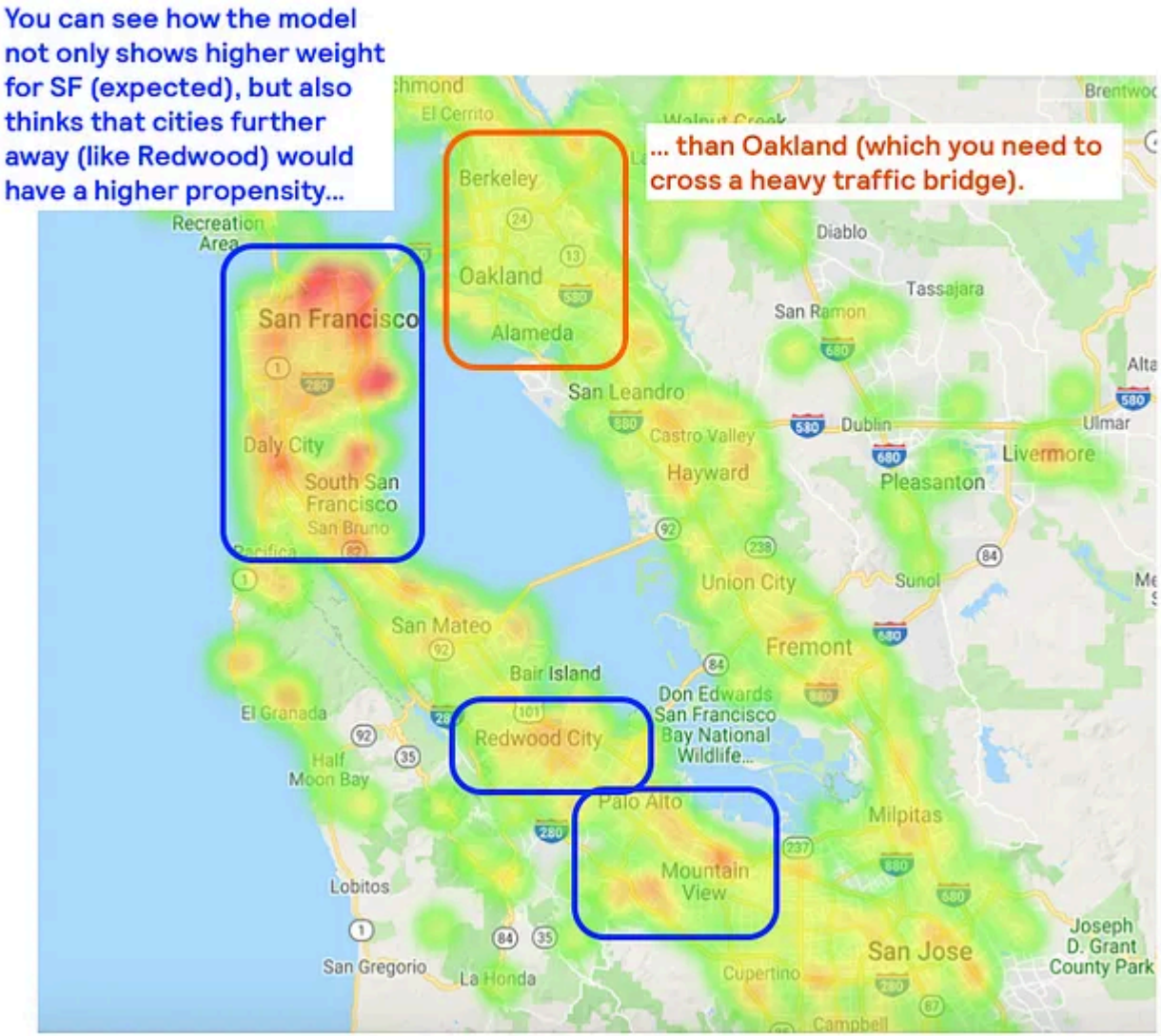


Figure 13: Location preference learnt for the query "San Francisco"

Why this works

Instead of treating location as a fixed coordinate, this method learns patterns in how users search. Someone searching for “San Francisco” might favour different neighbourhoods (Union Square vs. the Mission District vs. the Marina). The model discovers these preferences dynamically through back propagation.

💡 **Takeaway:** If you want to extract the juice out of NN and your data when applying deep learning; encode what users care about, not just what’s easiest to collect.

But hey, don’t feel bad if you don’t have this level of creativity. These guys at Airbnb are paid top $$$ and it must be because of something. The rest of us mortals can get inspiration from them.

Deep Learning explainability — what worked and what didn’t

Neural networks are powerful, but they are also black boxes (I mean a complex GLM model is also pretty black boxy, but definitely less than a big NN).

When Airbnb switched to deep learning for hotel ranking, they wanted to answer:

- *Why is the model making certain decisions?*

- *Which features actually matter?*

Here is what they tried.

Feature removal didn't reveal much

The first approach was simple: remove one feature at a time, retrain the model, and measure the impact.

❌ But, the neural network was resilient! It compensated for missing features by relying on correlated ones. The non-linear interactions captured by ReLU activations meant the model barely noticed when a single feature disappeared.

Permutation tests were helpful (but flawed)

I have always struggled with permutation tests, because they inherently create 'impossible scenarios' on the data. They also assume that the relationship between features is independent, but that is pretty much never true.

❌ Take hotel ranking, for example:

- **Price** is inherently tied to the **number of nights, rooms, guests, and hotel quality**.
- A **5-star hotel for 10 nights** will never have a price comparable to a **3-star budget listing**

- But, you can create these scenarios in a permutation test so you do get into these useless grey areas (and sometimes mislead interpretation).

✅ However, permutation tests did help Airbnb identify features that weren't important at all: *"If randomly permuting a feature didn't affect model performance at all, it was a good indication that the model was probably not dependent on it."*

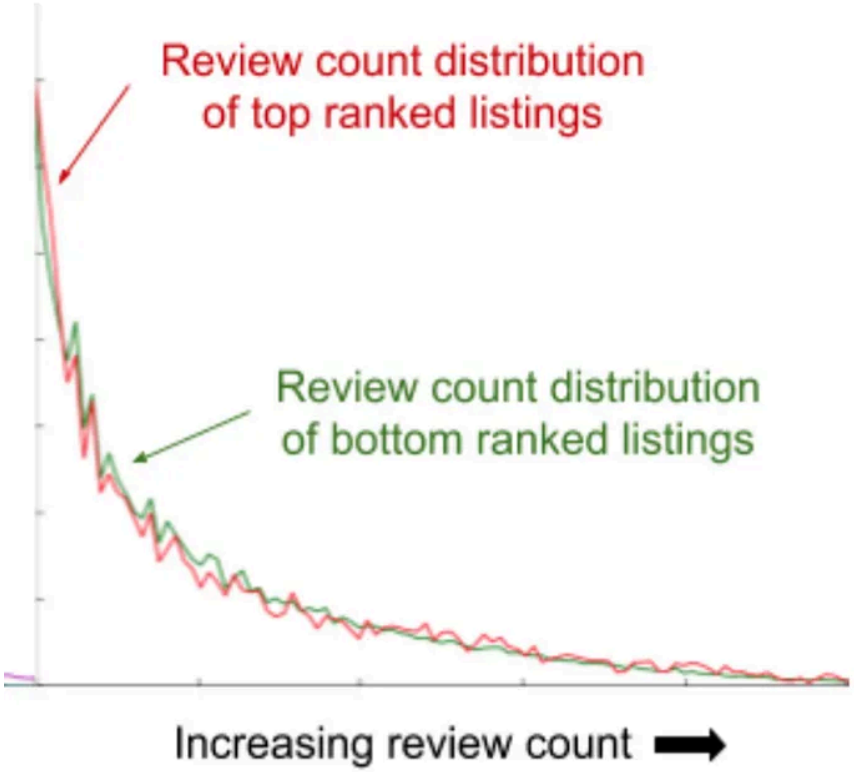
TopBot analysis: Airbnb's custom explainability method

This was something developed in-house. Nothing that I have heard of used anywhere else or open-sourced.

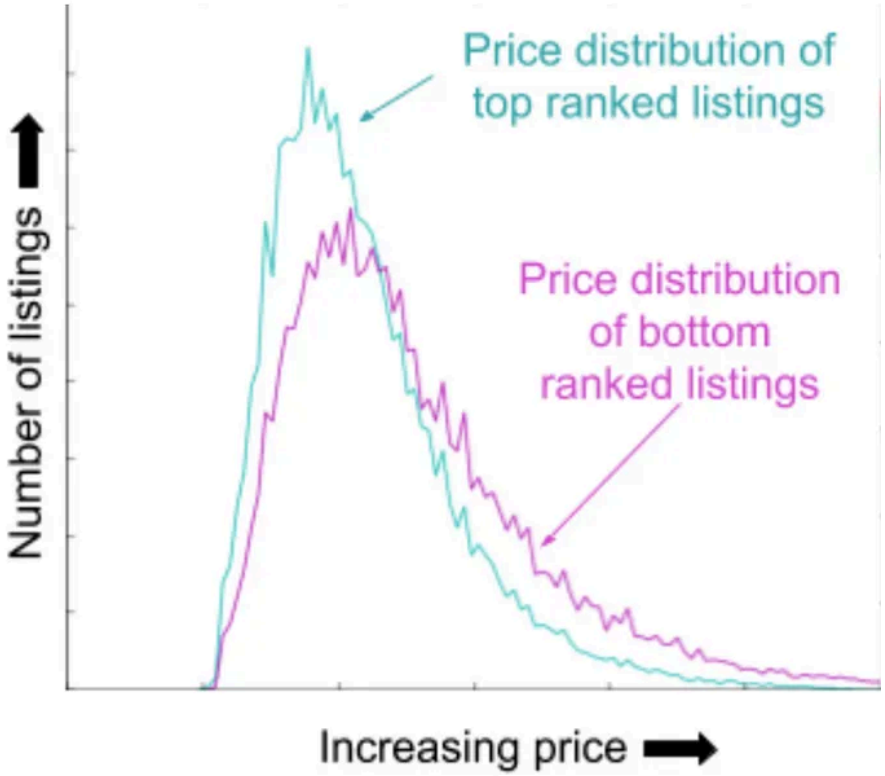
How it works:

- Rank a test set.
- Plot the distribution of a feature for top-ranked vs. bottom-ranked listings.
- If the distributions are similar, the feature isn't playing a major role.
- If the distributions are different, the model is using this feature to drive relevance.

For example, plotting review count distribution revealed how unimportant it was in ranking listings.



On the other hand, if the feature had a very different distribution, this would be an indication of how was the model using this feature to discriminate relevance. In this case, price clearly has an effect on how the neural network was pushing more expensive prices to bottom ranks.



As you can see, applying deep learning is one thing, but understand what happens when you apply it, is something completely different. I cannot stress the importance of trying to explain what your model is doing. Not only for your own benefit to keep improving model performance, but because stakeholders will surely see this either as a *'I trust the magic box'* or *'This is dark magic and unless I can understand, I will not trust it'*.

Summary

Neural networks are powerful, but making them work in production is a different challenge. Airbnb's journey showed that:

- **Start simple**. Deep learning isn't always better than traditional models right away.
- **Feature engineering still matters**. From normalisation to geo-location embeddings, the right transformations can make or break a model.
- **Explainability is hard, but necessary**. Techniques like permutation tests and Airbnb's own TopBot analysis help uncover what the model is really doing.

Deep learning is powerful, but the real magic isn't just in building models—it's in making them work at scale, explaining their decisions, and learning when to keep things simple. *That* is the real lesson from Airbnb's journey.

Now, I want to hear from you!

📢 What's your biggest challenge when applying deep learning in real-world projects?

- Do you struggle with feature engineering, explainability, or getting deep learning to outperform traditional models?
- Have you used permutation tests or built custom explainability techniques like Airbnb's TopBot analysis?

Drop your thoughts in the comments! I would love to hear what works (or doesn't) for you! 🚀

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

13 Likes • 8 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Andres Vourakis Future Proof Data Science 29 Mar 2025

Liked by Jose Parreño Garcia

Loved the breakdown Jose!

Building a NN that scales is for sure not an easy feat.

I've been considering it as a replacement for a Gradient Boosted Tree model I've been building at work for the past 5 months but which has reached its natural ceiling.

Lots to consider here...

LIKED (1) REPLY SHARE

1 reply by Jose Parreño Garcia

1 more comment...

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture